

Chapter 5. Package Management

In [Chapter 1](#), we briefly learned about Node's default package manager, npm. It's now time to take a deeper look and get comfortable finding, using, and creating packages for Node.

The term *package* is what the software world uses to describe a folder that contains code. In Node, that folder will also have a *package.json* file that describes the metadata and dependencies of the package.

The term *module* refers to a single file or a collection of related files that encapsulate a set of functionality. Modules allow developers to organize their code into separate and reusable units. A Node package often represents a single Node module, but some packages have more than one module.

A package usually refers to *external* code that a project depends on, but I think a better word to describe package code is *generic*. You can make pieces of your own code generic and extract them into a package that you can then use in many projects.

Introducing Package Management

If packages are just folders, why exactly do we need a *package manager* for them?

Keeping track of these package folders becomes challenging when there are many of them and when these packages depend on other packages. This is especially true for a team of developers working on the same Node project. Package management tools provide a simplified and systemic approach to handling the common tasks around packages. They provide simple commands to install, update, and remove packages, and to ensure

that a project has exactly what it needs to function correctly and similarly on all machines that are running it.

More importantly, package management tools can manage any conflicts among all the dependencies in the project, which are usually referred to as the *dependency tree*. It's a tree because a project has a main list of dependencies, and these main dependencies have their own dependencies, and so on. The term *transitive dependency* is often used to refer to all the dependencies in a project that are beyond the first level of the dependency tree.

NOTE

The npm tool has long been the default for managing packages and their dependencies in Node projects, but today there are a few alternatives, like Yarn, pnpm, and JSR. These alternatives to npm have their unique features and advantages. They often offer improvements on performance, disk space usage, and version management. This healthy competition has pushed npm to improve as well. In this book we're only covering npm, but you might end up using a different package manager. The basic concepts of package management are all similar, but the command interfaces and what happens behind the scenes are a bit different.

The term *npm* is mainly used to refer to the CLI (`npm` command) that ships with Node and provides tools to manage Node packages. There is also the [npm website](#), which hosts the public registry of many open source npm packages. The npm registry is like a big warehouse full of JavaScript packages, offering many options for common features and functionalities that you might need to add to your projects. For example, if you need your project to handle web requests, handle web sockets, or connect to a database, you don't need to build these features from scratch or deal with low-level code. You can download and use ready-made and often battle-tested generic solutions from a package registry and then build your custom needs on top of them.

But can these ready-made solutions be trusted? You need to be the judge of that, but many of these packages have already established the trust and respect of the JavaScript and Node communities. All npm packages

are open source, so you can do your own research. There are bad actors in the space, so pick your packages carefully, and keep an eye on their updates. Even a trusted package might be hacked, but looking at the source code changes and the activities around the code changes (like GitHub issues, pull requests, etc.) helps mitigate the risk.

TIP

The [npm registry](#) is the default registry for the `npm` command, but npm is highly configurable. You can, for example, configure it to use a different registry.

Adopting a systemic approach to managing package dependencies is essential for a team project. With npm, because all package dependencies are configured in the project's *package.json* file (which is shared among all developers), it becomes easy to set up a new environment or update an older one. All developers on the team use similar versions for the project packages, and when conflicts happen, they can be detected early.

Packages usually get updated often to fix bugs, add new features, and improve things overall. With a package manager, you're in control of how to handle these updates. To ensure compatibility and prevent conflicts, you can specify which exact versions of packages the project needs. You can also automate installing important security patches.

The npm project started with a small set of Node scripts to manage common tasks around folders that contain code for Node, and it has since evolved into a fully featured package manager that is useful not only for Node but for all JavaScript code, everywhere. If you browse the packages that are hosted on the npm registry, you'll find packages that are for Node and packages that are libraries and frameworks meant to be used in a browser or a mobile application. If you dig deep enough, you'll see examples of packages for many other environments where JavaScript can be executed.

WARNING

The npm registry has a lot of useless packages. Anyone can publish packages, and there is no quality control. Don't take the presence of a package on that registry as a trust signal. Always do your research, look at how the package is used in other open source projects, and preferably, inspect its code yourself.

One cool tool you can use to find actively maintained and popular packages in the Node ecosystem is the [Node.js Toolbox website](#). Top packages there are grouped by tasks, so, for example, you can see all your options for an email-delivery package or a file-upload package.

The npm Command

The npm CLI is a powerful one that supports many commands. To see usage instructions and the list of all the available commands, you can run

```
npm --help:
```

```
$ npm --help
```

```
npm <command>
```

Usage:

npm install	install all the dependencies in your project
npm install <foo>	add the <foo> dependency to your project
npm test	run this project's tests
npm run <foo>	run the script named <foo>
npm <command> -h	quick help on <command>
npm -l	display usage info for all commands
npm help <term>	search for help on <term>
npm help npm	more involved overview

All commands:

access, adduser, audit, bin, bugs, cache, ci, completion, config, dedupe, deprecate, diff, dist-tag, docs, doctor, edit, exec, explain, explore, find-dupes, fund, get, help, hook, init, install, install-ci-test, install-test, link,

ll, login, logout, ls, org, outdated, owner, pack, ping, pkg, prefix, profile, prune, publish, rebuild, repo, restart, root, run-script, search, set, set-script, shrinkwrap, star, stars, start, stop, team, test, token, uninstall, unpublish, unstar, update, version, view, whoami

Don't be overwhelmed by the number of commands you see here. You don't really need many of them. The commands you'll use often are `install` and `update`. You'll also probably use `run` commands like `start` and `test` and a handful of other commands depending on the type of project.

Most other commands you'll use infrequently. Here are a few highlights:

`npm create` ([Figure 5-1](#))

Sets up a new or existing npm package. This is an alias to the `init` command that we used in [Chapter 1](#). You can use it to create a *package.json* file or execute an *initializer* for packages that provide them (like ESLint or Vite). An initializer determines how a new application is configured and built, and it can do many other tasks as well.

```
TERMINAL
$ npm create vite@latest -- --template vanilla
Need to install the following packages:
create-vite@5.5.3
Ok to proceed? (y) y

> efficient-node@1.0.0 npx
> create-vite --template vanilla

✓ Project name: ... vanilla-test

Scaffolding project in /Users/samer/efficient-node/vanilla-test...

Done. Now run:

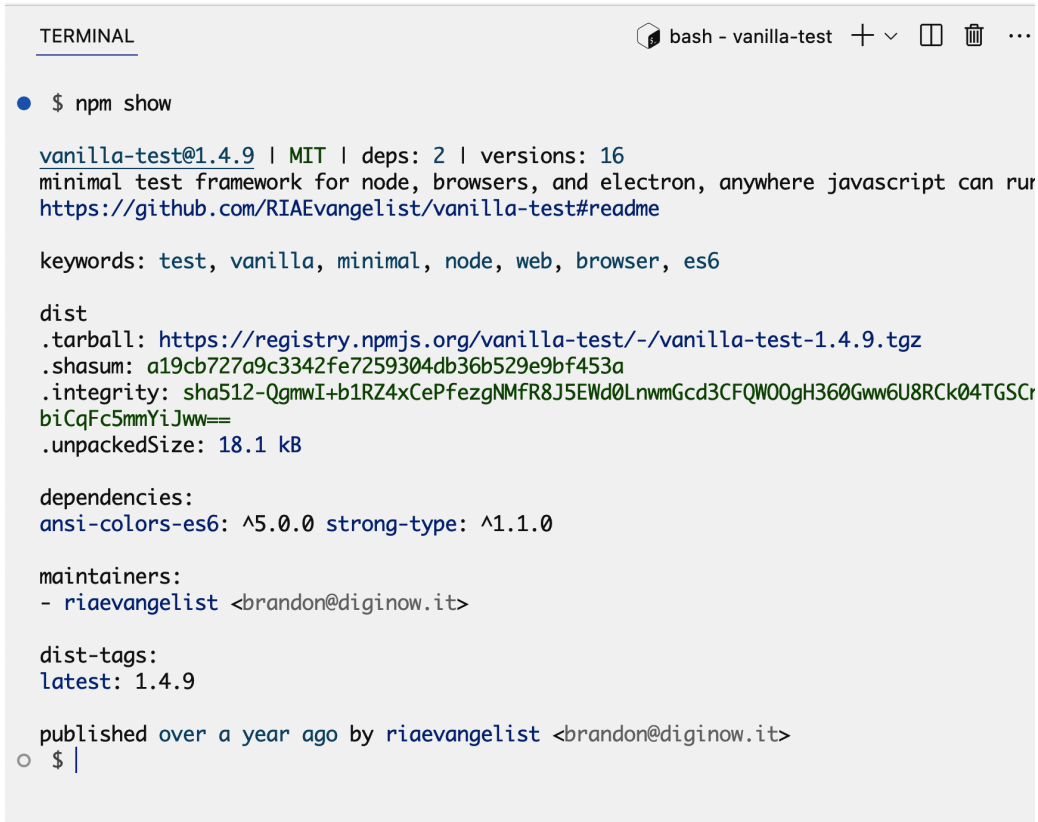
  cd vanilla-test
  npm install
  npm run dev

$ |
```

Figure 5-1. `npm create`

`npm show <package>` ([Figure 5-2](#))

Shows information about a package, such as what license it uses, how many dependencies it has, when it was published, and so on. You can use it without an argument to see information about the current package.

A terminal window titled 'TERMINAL' with a tab 'bash - vanilla-test'. The command '\$ npm show' has been executed. The output displays package metadata for 'vanilla-test@1.4.9', including its MIT license, 2 dependencies, 16 versions, a minimal test framework description, a GitHub link, keywords, distribution details (tarball, shasum, integrity, unpacked size), dependencies (ansi-colors-es6, strong-type), maintainers (riaevangelist), dist-tags (latest: 1.4.9), and publication info (published over a year ago by riaevangelist).

```
TERMINAL bash - vanilla-test + v [] [trash] ...

● $ npm show

vanilla-test@1.4.9 | MIT | deps: 2 | versions: 16
minimal test framework for node, browsers, and electron, anywhere javascript can run
https://github.com/RIAEvangelist/vanilla-test#readme

keywords: test, vanilla, minimal, node, web, browser, es6

dist
.tarball: https://registry.npmjs.org/vanilla-test/-/vanilla-test-1.4.9.tgz
.shasum: a19cb727a9c3342fe7259304db36b529e9bf453a
.integrity: sha512-QgmwI+b1RZ4xCePfezgNMfR8J5EWd0LnwmGcd3CFQW00gH360Gww6U8Rck04TGSCr
biCqFc5mmYiJww==
.unpackedSize: 18.1 kB

dependencies:
ansi-colors-es6: ^5.0.0 strong-type: ^1.1.0

maintainers:
- riaevangelist <brandon@diginow.it>

dist-tags:
latest: 1.4.9

published over a year ago by riaevangelist <brandon@diginow.it>
○ $ |
```

Figure 5-2. npm show

`npm search <search terms>` ([Figure 5-3](#))

Searches the npm registry for packages based on the provided search query. For example, try `npm search lodash`.

```
TERMINAL
bash + v [ ] [ ] ...

• $ npm search lodash
lodash
Lodash modular utilities.
Version 4.17.21 published 2021-02-20 by bnjmnt4n
Maintainers: mathias jdalton bnjmnt4n
Keywords: modules stdlib util
https://npm.im/lodash

@types/lodash
TypeScript definitions for lodash
Version 4.17.10 published 2024-10-03 by types
Maintainers: types
https://npm.im/@types/lodash

lodash.merge
The Lodash method `_.merge` exported as a module.
Version 4.6.2 published 2019-07-10 by jdalton
Maintainers: jdalton mathias
Keywords: lodash-modularized merge
https://npm.im/lodash.merge
```

Figure 5-3. npm search

`npm list <package>` (*Figure 5-4*)

Displays a tree-like view of installed packages and their dependencies, along with their versions. You can use it on a single package or for the entire application. A common alias is `npm ls`. Try it with a `--depth=1` option to see the first level of transitive dependencies.

```
TERMINAL
bash + v [ ] [ ] ...

• $ npm ls --depth=1
efficient-node@1.0.0 /Users/samer/efficient-node
├── @eslint/js@9.11.1
│   └── eslint@9.11.1
│       ├── @eslint-community/eslint-utils@4.4.0
│       ├── @eslint-community/regexpp@4.11.1
│       ├── @eslint/config-array@0.18.0
│       ├── @eslint/core@0.6.0
│       ├── @eslint/eslintrc@3.1.0
│       ├── @eslint/js@9.11.1 deduped
│       ├── @eslint/plugin-kit@0.2.0
│       ├── @humanwhocodes/module-importer@1.0.1
│       ├── @humanwhocodes/retry@0.3.0
│       ├── @nodelib/fs.walk@1.2.8
│       ├── @types/estree@1.0.6
│       ├── @types/json-schema@7.0.15
│       ├── ajv@6.12.6
│       ├── chalk@4.1.2
│       ├── cross-spawn@7.0.3
│       ├── debug@4.3.7
│       └── escape-string-regexp@4.0.0
```

Figure 5-4. npm list

`npm link`

Creates a symbolic link between a package in your local filesystem and a package installed under *node_modules* (or globally). This al-

allows you to develop and test packages locally without the need for publishing or reinstalling.

```
npm cache clean
```

Clears the npm cache, which can help resolve certain installation issues or outdated package versions.

```
npm publish
```

Publishes your package to the npm registry, making it available for others to install. We'll see an example of that later in the chapter.

You can get further details and instructions on any npm command using `npm <command> -h`. Here's the help summary for the `npm install` command:

```
$ npm install -h
Install a package

Usage:
npm install [<@scope>/]<pkg>
npm install [<@scope>/]<pkg>@<tag>
npm install [<@scope>/]<pkg>@<version>
npm install [<@scope>/]<pkg>@<version range>
npm install <alias>@npm:<name>
npm install <folder>
npm install <tarball file>
npm install <tarball url>
npm install <git:// url>
npm install <github username>/<github project>

Options:
[-S|--save|--no-save|--save-prod|--save-dev|--save-optional|..
...

aliases: add, i, in, ins, inst, ...
```

As you can see, we can use the `install` command in many ways and with many options. It also has many aliases.

You don't need to remember all of the usage ways and options, but doing a quick scan for later reference is certainly helpful.

TIP

This is actually the summarized version of the install help page. You can see the full help page using `npm help install`.

Here are a few challenges for you to figure out from the help text of `npm install`:

- Install a package that's hosted under a scope. An npm scope is a way to group related packages under a specific namespace or organization. An example scope is `@babel`. An example package under that scope is `core`.
- Install a package directly from GitHub. Try to install `lodash` from GitHub. To verify, look at the `dependencies` section of the project's `package.json` file—`lodash` should have a `github` label.
- Install a package globally to make it available to any Node project on the machine. This option is commonly used for command-line tools. For example, you can install the ESLint package globally using npm, and that will make the `eslint` command available everywhere.

TIP

Avoid installing npm packages globally unless you really need to. Installing packages globally reduces the modularity of your projects and can lead to version conflicts between different projects. It can also cause your projects to behave inconsistently across different environments.

Semantic Versioning

The `npm update` command can be used to update packages listed in `package.json` to their latest version (as constrained in the file). To under-

stand that, we first need to learn about *semantic versioning* (or *SemVer* for short).

SemVer is used by npm when it's time to update packages. Every package has a version. A version is one of the required pieces of information about a package, and it's usually written with the SemVer format. For example, when we installed the `lodash` package in [Chapter 1](#), the line that was added to the `package.json` dependencies section was as follows:

```
"lodash": "^4.17.21"
```

The `4.7.21` part is the SemVer string, and it's basically a simple contract between a package author and the users of that package. When that number gets bumped up to release a new version of the package, the SemVer communicates how big of a change that new release will be to the package.

The first number, which is called the *major* number, is used to communicate that breaking changes happened in a release. Those are changes that will require users to change their code to make it work with the new release. The next time that happens for `lodash`, it'll be released with a SemVer string that begins with `5` instead of `4`.

The second number, which is called the *minor* number, is used to communicate that new features were added in a release but older features should still work as is. A minor version release might also include warnings about future deprecations and API changes. Minor versions updates should still be backward-compatible and safe for users to update, without needing to make any changes to their projects.

The last number, which is called the *patch* number, is used to communicate that the release contains only bug fixes and security improvements. It should not introduce any new features or breaking changes.

You'll often see special characters before the SemVer numbers. These special characters represent a range of acceptable versions and are put to use when you instruct npm to update your dependency tree.

For example, the tilde (`~`) character means that an update can install the most recent patch version (remember, patch is the third number). The caret (`^`) character is a more relaxed constraint that means that an update can install the most recent minor version. If we update the `lodash` package while its version string is `^4.17.21`, it'll try to find the latest version that begins with the `4` major number. So it might install a `4.19.1` package, but it will not install a `5.1.2` package.

Other special characters are `=`, `>`, `>=`, `<`, `<=`. If no special character is used, it means the version to be used should always be the exact one that's specified by the SemVer string. When you install an npm package, you can use the `--save-exact` (or `-E`) to instruct npm to save the dependency as an exact version without special characters.

Instead of a version string, an asterisk (`*`) can be used to mean the latest version available.

Another way to specify the version constraint is with an `x` in the string. For example, a `4.x` version string means any version that begins with a `4`. A `4.17.x` string means any version that begins with `4.17`.

You can also manually specify a range using the `-` character, for example: `4.15.0 - 4.17.0`.

TIP

For more details on version strings and for an interactive way to test them, check out the [npm SemVer calculator](#). You can enter a version string for a particular npm package to see all the available versions constrained by that string.

I think SemVer is great. Responsible npm developers should respect it when they release new versions of their code, but it's good to treat what it communicates as a promise rather than a guarantee because even a patch release might leak breaking changes through its own dependencies. A minor version, for example, might introduce new elements that conflict with elements you previously thought were OK to use. Testing your code

is the only way to provide some form of guarantee that it's not broken after an update.

Updating and Removing Packages

When the packages in your project have available updates, you can issue the `npm update <package-name>` command to update a single package, or the `npm update` command to update all the packages in the dependency tree.

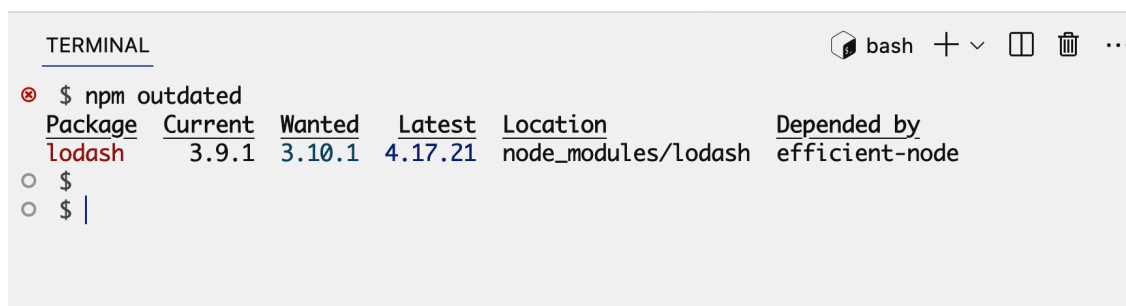
Let's simulate a case where an update is going to happen by installing an older version of `lodash`. To do that, we just specify the exact version we are interested in by adding it after an `@` character:

```
$ npm install lodash@3.9.1
```

You can verify which version npm installed using the `npm ls` command. It should be `3.9.1`.

Now take a look at `package.json` and note how the version string starts with a `^` character. This permits npm to update the package to the latest minor version available.

To see what version will be installed using the `npm update` command, you can first run the `npm outdated` command. It'll list all packages, and if any of them has a valid update (permitted by the version strings constraints), the updated version will be listed under the `Wanted` column. As shown in [Figure 5-5](#), the output will also include the latest version.



```
TERMINAL
ⓧ $ npm outdated
Package   Current  Wanted  Latest  Location  Depended by
lodash    3.9.1    3.10.1  4.17.21 node_modules/lodash efficient-node
$
$ |
```

Figure 5-5. `npm outdated`

Now because of the `^` constraint, the `Wanted` version in this case will be `3.10.1`. That was the last `lodash` version released under the `3` major branch.

If you change the `^` into the more strict `~` and run the `npm outdated` command, the `Wanted` version will be `3.9.3`. That was the last `lodash` version released under the `3.9` minor branch.

If you change `~` into `>` and run the `npm outdated` command, the `Wanted` version will match the latest one.

The `outdated` command is like a dry run for you to verify what packages will be updated. It does not perform the update. To update, you run the `npm update` command.

Experiment with the `outdated`, `update`, and `ls` commands with a package like ESLint that has its own dependencies. Install an older version of that as well, for example:

```
$ npm i eslint@8
```

Note the version I use here. That `8` is the major version, and the syntax means install the latest ESLint version that begins with `8`. See what version was installed with the `npm ls` command.

What happens if you change the version string in `package.json` to something older? For example, change the ESLint version string to `~7.32.0`. Since that constraint specifies something older than what you currently have installed, running the `npm update` command will actually downgrade the ESLint package. Verify that with `npm ls`.

The `update` command will update all dependencies, including transitive ones, based on the version strings constraints specified in the `package.json` files of the packages that depend on them.

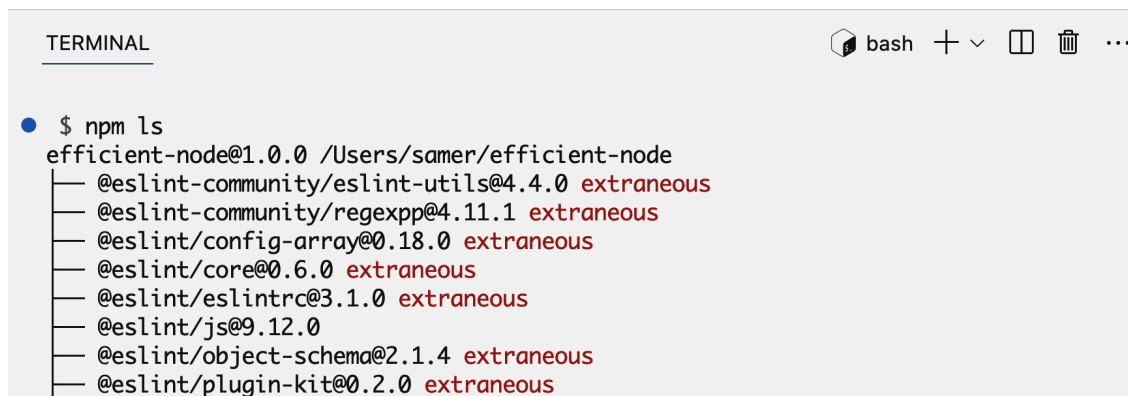
To make the `outdated` command show all the dependencies to be updated, run it with the `-a` flag:

```
$ npm outdated -a
```

Let's say that we decided we no longer want to use the ESLint package. You can remove it from *package.json* manually, but that will not remove it from *node_modules*. To remove it from both *package.json* and *node_modules*, you can run the `npm uninstall <package-name>`. The `uninstall` command is the better way here.

However, if someone on the team used the `uninstall` command and you pulled that code change, all you're seeing is the line being removed from *package.json*. The *node_modules* folder is not usually shared in source control repos. You'll need to run npm commands to sync your *node_modules* folder with the updates in *package.json*.

To simulate that, remove the `eslint` line from *package.json*. You now have packages that are installed but no longer needed (according to *package.json*). If you run the `npm ls` command now, it'll list these packages with an "extraneous" label next to them, as shown in [Figure 5-6](#).

A terminal window titled "TERMINAL" with a "bash" icon and window controls. It shows the command "\$ npm ls" and its output. The output lists the project "efficient-node@1.0.0" and its dependencies. Most dependencies are marked as "extraneous" in red text, including "@eslint-community/eslint-utils@4.4.0", "@eslint-community/regexpp@4.11.1", "@eslint/config-array@0.18.0", "@eslint/core@0.6.0", "@eslint/eslintrc@3.1.0", "@eslint/object-schema@2.1.4", and "@eslint/plugin-kit@0.2.0". The dependency "@eslint/js@9.12.0" is not marked as extraneous.

```
TERMINAL bash + - [ ] [X] ...
● $ npm ls
efficient-node@1.0.0 /Users/samer/efficient-node
├── @eslint-community/eslint-utils@4.4.0 extraneous
├── @eslint-community/regexpp@4.11.1 extraneous
├── @eslint/config-array@0.18.0 extraneous
├── @eslint/core@0.6.0 extraneous
├── @eslint/eslintrc@3.1.0 extraneous
├── @eslint/js@9.12.0
├── @eslint/object-schema@2.1.4 extraneous
└── @eslint/plugin-kit@0.2.0 extraneous
```

Figure 5-6. Extraneous packages

To remove all unused packages from the project, you can use the `npm prune` command:

```
$ npm prune

removed 84 packages, and audited 4 packages in 2s

1 package is looking for funding
  run `npm fund` for details
```

found 0 vulnerabilities

Now if you run the `npm ls` command again, there should not be any extraneous packages.

To ensure that a project's dependencies are in sync with changes in *package.json*, whenever you pull new code and notice changes to *package.json*, run both the `prune` and `install` commands.

However, the `npm install` command will always install the latest version of a package as permitted by version string constraint. That means in the time between a dependency being added by one developer and another developer pulling the code to install it, a new version of that dependency might have been released, and if the version string specified in *package.json* allows it, `npm install` will install that new version, which is different from the one that's installed on the machine that added the dependency in the first place.

That's why npm automatically maintains another file at the root of the project, the *package-lock.json* file. The purpose of that file is to lock versions of packages so that all project developers use the exact same versions of all the packages. This is true for both direct dependencies and transitive ones.

Every time a dependency is added, updated, or removed, npm will modify the *package-lock.json* file to describe the entire tree of dependencies (direct and transitive), along with what exact versions to install.

Because the *package-lock.json* file should be part of the project Git repository for others to use it, its change history can be used to go back to previous states of exactly what was under the *node_modules* folder.

The *package-lock.json* file is also used by npm to optimize its operations.

Creating and Publishing Packages

Let's create and publish a simple npm package that provides a function named `printInFrame`. That function takes a string argument and outputs that string within a frame made of `*` characters. Let's name the package `print-in-frame`.

Here's an example of how we'd use it:

```
import printInFrame from 'print-in-frame';

printInFrame('Hello World');
```

Running this code should output the following:

```
*****
* Hello World *
*****
```

First, make a new folder to host this package code. The name of the folder usually matches the name of the package (although that's not a requirement):

```
$ mkdir print-in-frame & cd print-in-frame
```

The next step is to make this empty folder into an npm package. We do that by adding a *package.json* file. We can use `npm init` for that:

```
$ npm init
```

Answer the questions or use the default answers. After the file is created, manually add the `'type': 'module'` to instruct Node that this project will exclusively use ES modules.

Open up your code editor and create an `index.js` file in the root of the project. To implement the `printInFrame` function, we need to read the length of the text and use that to print a set of `*` characters before and after. This can be done in many ways. Here's what I did:

```
// In index.js
import times from 'lodash.times';

const printInFrame = (text) => {
  const frameWidth = text.length + 4; // 2 stars + 2 spaces

  let textToPrint = '';

  times(frameWidth, () => (textToPrint = textToPrint + '*'));

  textToPrint = textToPrint + '\n' + '* ' + text + ' *' + '\n';

  times(frameWidth, () => (textToPrint = textToPrint + '*'));

  console.log(textToPrint);

  return textToPrint;
};

export default printInFrame;
```

I made the function depend on `lodash.times`, which provides a function that can repeat a block of code any number of times. I used that to prepare the frame header and footer lines. You need to `npm install lodash.times`.

To use the `print-in-frame` package in a Node project, we need to install it. We can actually install it directly from the filesystem. For example, if we're in a Node project that's in the same directory as the `print-in-frame` folder, in that other Node project we can do the following:

```
$ npm install ../print-in-frame
```

While this works OK, when you share your code with others, you'll have to share the *print-in-frame* folder as well. To keep them separate, we'll need to use an npm registry and publish the package there.

If you want to publish your package in the [npm registry](#), you need to have an account there. Then you can use the `npm login` command to authenticate your local npm client with your account. It'll ask you for your username and password.

Since the package name is unique at the npm registry, to avoid conflict, add a prefix to your package name. I changed the `name` property in *package.json* to `samer-print-in-frame`. While you're there, add a description to the package as well. It's optional, but it makes the package easier to discover.

When you're ready, run the `npm publish` command. If everything works, your package will be available on the npm registry website (use the UI search there to find it). You can also use the `npm search` command to find it.

With the package published, install it in your main Node project with `npm install PREFIX-print-in-frame`, replacing `PREFIX` with the prefix that you used.

Now look at the output of `npm ls`. You should see two new dependencies: `PREFIX-print-in-frame` and `lodash.times`.

To make updates to your package and test them in a project before you publish a new version, you can use the `npm link` command to temporarily make a project use a local package rather than the one installed through the registry. In the *print-in-frame* folder, run `npm link`; then in the main project folder, run `npm link PREFIX-print-in-frame`.

Now you can make changes to your local package folder and test them in your main project. Once you're done, you can increment the `version` property in the *package.json* file under your package and run `npm publish` again.

NOTE

I used ES modules for `print-in-frame`. This means it can be used only under projects that use ES modules. If you want to create a package that can be used in any Node project, you'll need to create a CommonJS version as well. You can use tools like Babel or TypeScript to automate tasks like these.

npm Run Scripts

Scripts are a feature in npm that enables developers to easily perform (or automate) common tasks like building, testing, and deploying applications.

You can define a run script under the `scripts` section in *package.json*. When you run the `npm init` command, it'll include an example run script:

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1"  
},
```

You can use that `test` script by running `npm run test`. A few common run script names (like `test`, `start`, `stop`) have a shortcut alias as well. You can run the `test` script here with just `npm test`.

TIP

If you run the `npm run` command without any arguments, it'll list all defined scripts under the project.

The sample `test` script just outputs an error message, but note how it used shell commands like `echo` and `exit`. You can use any of the shell commands available on your machine. For example, try a script to `ls -al` or to `npm ls | grep 'extraneous'`. The latter is a good example of how a common project task can be simplified into a run script and docu-

mented for other team members who don't know about it. What's a good intuitive name for that task? Maybe `list-unused-packages` :

```
"scripts": {  
  ...  
  "list-unused-packages": "npm ls | grep 'extraneous'"  
},
```

Now a developer who does not know about this extraneous label can look at this run script and figure out how to list any unused packages in the project. They just need to `npm run list-unused-packages` .

This becomes more important when you publish packages for other teams to use. The best place to communicate to developers how to use a package is in npm run scripts.

Run scripts help developers automate running tasks. First, if you need to run something repeatedly for the project—for example, run all integration tests—you'll have a simple and intuitive way of doing it, rather than trying to figure out the exact command every time. More importantly, an npm run script will make running this task consistent among all developers. All developers should be using the exact same command to run all integration tests. Even more importantly, if the way to run all integration tests needs to change, instead of manually announcing this change in a chat channel, you can communicate it with a change to *package.json*, which is forever kept in the project's Git history.

You can even make the automation official and adopt a way to run tasks automatically before or after other tasks. For example, I often forget to run `npm prune && npm install` after pulling new code and trying to run all tests. You can use an npm run script to automatically run the pruning and installing every time you run the tests.

To do that, you can define script names using a `pre` or `post` prefix. For this example, we can define a `pretest` script to prune/install:

```
"scripts": {  
  ...  
  "pretest": "npm prune && npm install"  
},
```

With that special script in place, every time you run `npm test`, the `prune` / `install` commands will be executed before running the tests.

This works with any script name. If you have `dosomething` script name, you can define the `predosomething` and `postdosomething` scripts to execute tasks before or after you run `dosomething`.

This is great for many use cases. You can, for example, automate running tests, formatting/linting, or generating documentations every time you try to push new code to its repo.

One other cool thing about npm run scripts is that they'll execute any command-line tools installed under the project. You don't need to explicitly specify the path to these commands.

For example, run `npm i eslint` under the project to install the `eslint` command-line. Now, if you're in the project folder and you try to execute the `eslint` command, it will not be available. That command is somewhere under the `node_modules` folder, but npm does not make it globally available. However, npm run scripts recognize the commands available under `node_modules`. To test that, add the following script:

```
"scripts": {  
  ...  
  "lint": "eslint"  
},
```

Now you can `npm run lint` and npm will find the `eslint` command and execute it. You can even include arguments, and npm will pass them to what you're executing, but since npm commands also accept arguments, you'll need to use a `--` separator to pass arguments to the exe-

cuted script. For example, to see the help page for `eslint`, this is what you need to do:

```
$ npm run lint -- --help
```

Executing the same line without the `--` arguments separator will output the help page for the `run-script` npm command. Any arguments you pass before the `--` separator will be used by the `run-script` npm command.

TIP

I named the script `lint` (instead of `eslint`) intentionally. Generic names are better under npm scripts. Maybe in the future we'll use something other than `eslint` to lint. Changing a run script name might break things in the future, especially automated tasks.

The npx Command

Another option to execute a command-line tool that's installed in a project is `npx` (which stands for *node package execute*). For example, running `npx eslint --help` will *always* work, even if the ESLint package is not installed in the project.

If a package is installed, the `npx` command will use the local folder under `node_modules`. If the package is not installed locally, `npx` will automatically download a temporary copy of the package and use it to execute the package.

Test it with `eslint`. Uninstall it with `npm uninstall eslint`, then run `npx eslint --help` to see how `npx` will download and then run the command.

Just like `npm`, you can use `npx` with specific versions. For example, let's say that you need to find out which of the `eslint` options (which you can

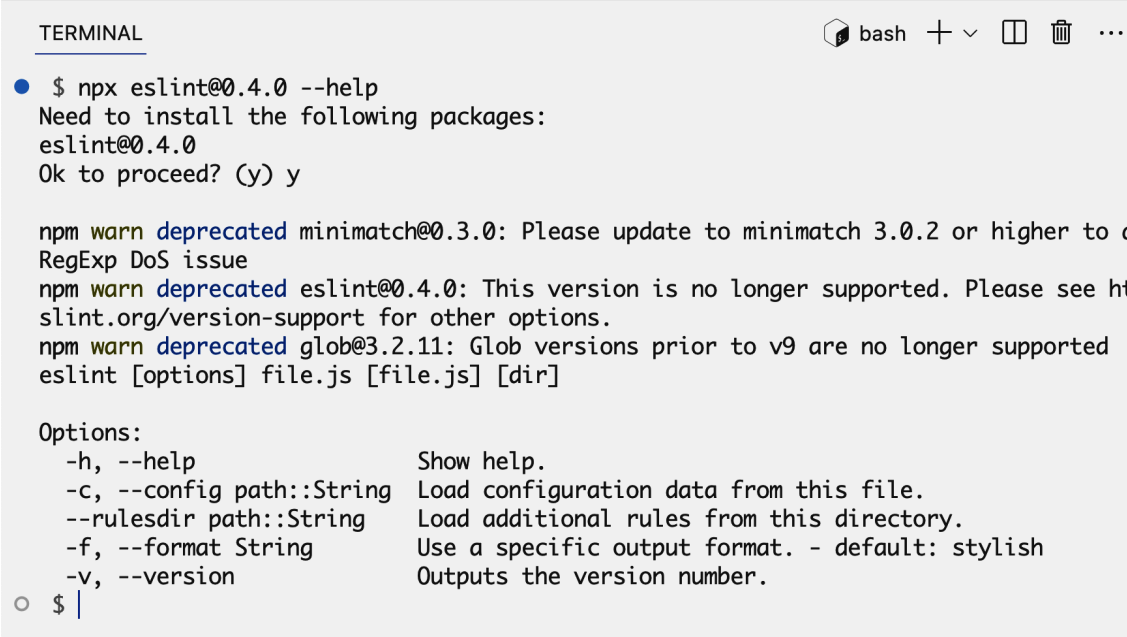
see in the help page) existed early on, since the first available version of `eslint`.

You can use the `npm show` command to find out the earliest available version of `eslint`:

```
$ npm show eslint versions
```

When I tested this command, the earliest version of `eslint` was 0.4.0. Note that an earlier version might have been available, but the maintainers of `eslint` decided to purge it from the registry.

To see the help page of the 0.4.0 `eslint` command, you can run `npx eslint@0.4.0 --help`, as shown in [Figure 5-7](#).



```
TERMINAL
$ npx eslint@0.4.0 --help
Need to install the following packages:
eslint@0.4.0
Ok to proceed? (y) y

npm warn deprecated minimatch@0.3.0: Please update to minimatch 3.0.2 or higher to c
RegExp DoS issue
npm warn deprecated eslint@0.4.0: This version is no longer supported. Please see ht
eslint.org/version-support for other options.
npm warn deprecated glob@3.2.11: Glob versions prior to v9 are no longer supported
eslint [options] file.js [file.js] [dir]

Options:
  -h, --help                Show help.
  -c, --config path::String Load configuration data from this file.
  --rulesdir path::String   Load additional rules from this directory.
  -f, --format String        Use a specific output format. - default: stylish
  -v, --version              Outputs the version number.
$ |
```

Figure 5-7. Using `npx` with a version

The `npx` command is commonly used to bootstrap a project from a template. An example of a package that can be used that way is `create-react-app`. You can use it through the `npx` command to generate a working React application using one of the many supported templates:

```
$ npx create-react-app your-app-name-here
```

Not only will this download a temporary copy of the `create-react-app` package, it'll then recognize that this is a *generator* package with a default command to create a project. It'll execute that default command.

Generator packages can even have multiple commands. Checkout the help page for the `@vue/cli` generator package to see an example of that:

```
$ npx @vue/cli --help
```

Summary

A package manager like npm is an important part of working on a Node project. It introduces a simple and standard way to deal with project external dependencies and keep them updated, consistent, and conflict-free.

With npm, packages are hosted on a public registry, and the `npm` command is configured to work with that registry. Related npm commands like `install`, `update`, `search`, and more also work with that registry.

The `package.json` and `package-lock.json` files are automatically modified by npm every time there is a change to the project dependency tree. These files store what versions of packages are installed and what range of versions should be used when updating packages.

In addition to the `npm` command, there's also an `npx` command that can be used to execute local or remote command-line tools.

In the next chapter, we'll explore how to work with streams in Node to efficiently handle large amounts of data without exhausting the process.